



VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Chapter 3: SELECTION STRUCTURE

September 2025

Content

-
- | | |
|---|---|
| 1 | Program Flow of Control |
| 2 | Boolean data type and boolean expressions |
| 3 | The <i>if-else</i> statement |
| 4 | Nested <i>if</i> statement |
| 5 | <i>switch</i> statement |
-

Content

1 Program Flow of Control

2	Boolean data type and boolean expressions
---	---

3	The <i>if-else</i> statement
---	------------------------------

4	Nested <i>if</i> statement
---	----------------------------

5	<i>switch</i> statement
---	-------------------------

Program Flow of Control

- The flow of control means the order in which a program's statements are executed.
- Unless directed otherwise, the normal flow of control for all programs is **sequential**.
- Selection, repetition and function invocation structures permit the flow of control to be **altered** in a defined way.
- This chapter introduces selection structures in C++. Chapter 4 discusses repetition and Chapter 6 focuses on functions.

Content

1 Program Flow of Control

2 Boolean data type and boolean expressions

3 The *if-else* statement

4 Nested *if* statement

5 *switch* statement

The *bool* Data Type

- As specified by the ANSO/ISO standard, C++ has a built-in **Boolean** data type, `bool`, containing the two values **true** and **false**.
- The actual values represented by the **bool** values, true and false, are the integer values **1** and **0**, respectively.
- Example 3.2.1:

```
#include <iostream>
using namespace std;
int main() {
    bool t1, t2;
    t1 = true; t2 = true;
    cout << "The value of t1 is " << t1
         << "\n and the value of t2 is " << t2 << endl;
    return 0;
}
```

Comparison Operators

- Comparison Operators: are used to compare two operands for equality or to determine if one numeric value is greater than another.
- A Boolean value of true or false is returned after two operands are compared. C++ uses a nonzero value to represent a true and a zero value to represent a false value.

Operator	Description	Example
<code>==</code>	equal	<code>a == 'y'</code>
<code>!=</code>	not equal	<code>m != 5</code>
<code>></code>	greater than	<code>a * b > 7</code>
<code><</code>	less than	<code>b < 6</code>
<code><=</code>	less than or equal	<code>b <= a</code>
<code>>=</code>	greater than or equal	<code>c >= 6</code>

Logical operators

- Logical operators are used for creating more complex conditions. Like comparison operators, a Boolean value of true or false is returned after the logical operation is executed.

Operator	Description
&&	AND
	OR
!	NOT

- Truth table:

A	B	A && B	A B	!A	!B
true	true	true	true	false	false
true	false	false	true	false	true
false	true	false	true	true	false
false	false	false	false	true	true

Logical operators (cond.)

- Example:
 - Assume: age = 45, term = 5 , i = 3, j = 3, a = 2, b = 5, complete = false
 - (age > 40) && (term < 10)
 - true
 - (age > 40) || (term < 10)
 - true
 - !(age > 40)
 - false
 - (i == j) || (a < b) || complete
 - true

Short-circuit evaluation

- **Short-circuiting** is a programming concept in which the **compiler skips** the execution or evaluation of some sub-expressions in a logical expression.
 - Evaluation stops as soon as the value of the expression is determined.
- Example 3.2.2: `(a == b) || (c == d) || (e == f)`
 - If the expression `(a == b)` is **true**, then `(c == d)` and `(e == f)` are not evaluated because the expression's result has already been determined.
- Example 3.2.3: `(a == b) && (c == d) && (e == f)`
 - If the expression `(a == b)` is **false**, the compiler will skip evaluating other sub-expressions.

Operator precedence

- The relational and logical operators have a hierarchy of execution similar to the arithmetic operators.

Level	Operator	Associativity
1	! unary - ++ --	Right to left
2	* / %	Left to right
3	+ -	Left to right
4	< <= > >=	Left to right
5	== !=	Left to right
6	&&	Left to right
7		Left to right
8	= += -= *= /=	Right to left

Example 3.2.4

- Assume the following declarations:

```
char key = 'm';
```

```
int i = 5, j = 7, k = 12;
```

```
double x = 22.5;
```

Expression	Equivalent	Value	Interpretation
<code>i + 2 == key - 1</code>	<code>(i + 2) == (key - 1)</code>	0	false
<code>'a' + 1 == 'b'</code>	<code>('a' + 1) == 'b'</code>	1	true
<code>25 >= x + 1.0</code>	<code>25 >= (x + 1.0)</code>	1	true
<code>k - 1 > 20</code>	<code>(k - 1) > 20</code>	0	false

Order of evaluation

- The following compound condition is evaluated as:

$(6*3 == 36/2) \ || \ (13 < 3*3 + 4) \ \&\& \ !(6-2 < 5)$

$(18 == 18) \ || \ (13 < 9 + 4) \ \&\& \ !(4 < 5)$

1 || $(13 < 13)$ && ! 1

1 || 0 && 0

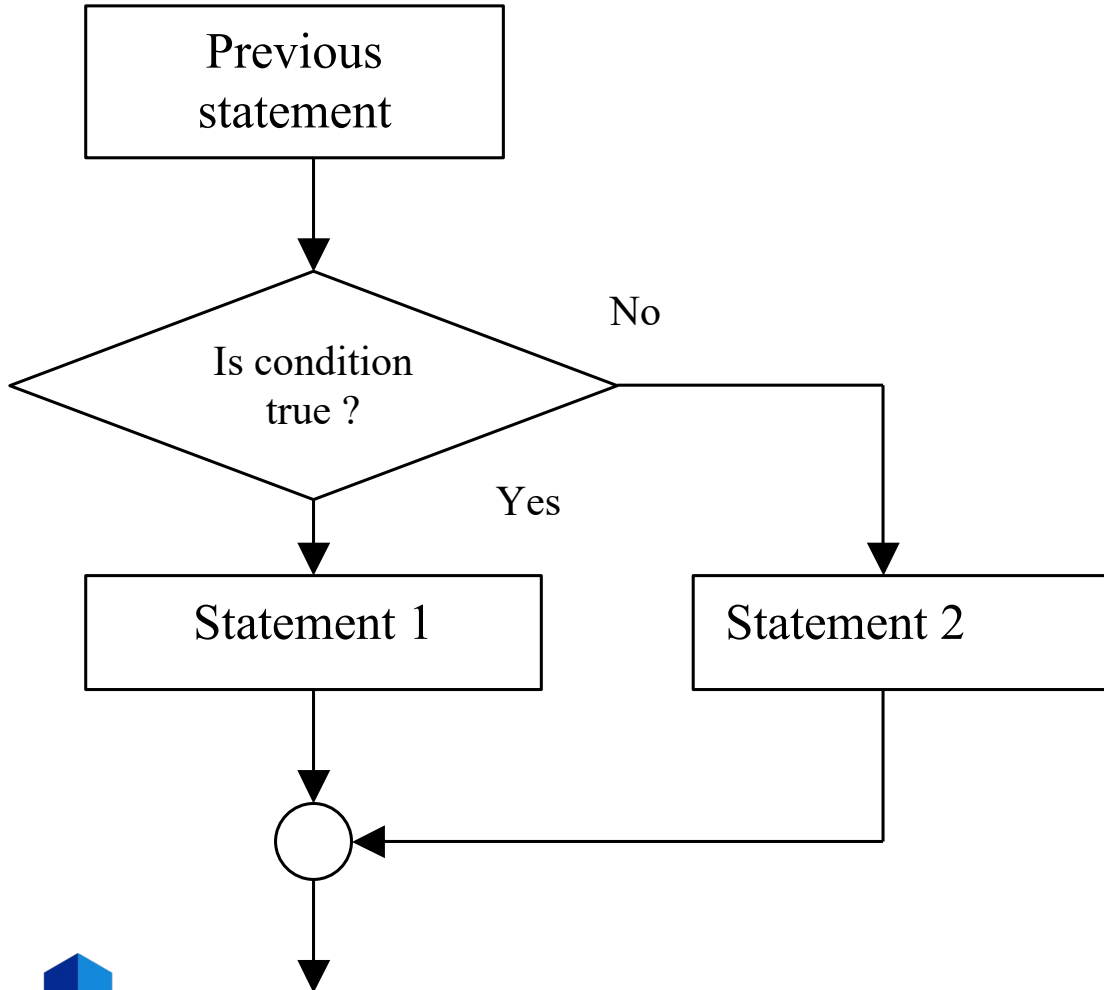
1 || 0

1

Content

-
- | | |
|---|---|
| 1 | Program Flow of Control |
| 2 | Boolean data type and boolean expressions |
| 3 | The if-else statement |
| 4 | Nested <i>if</i> statement |
| 5 | <i>switch</i> statement |
-

The *if-else* statement

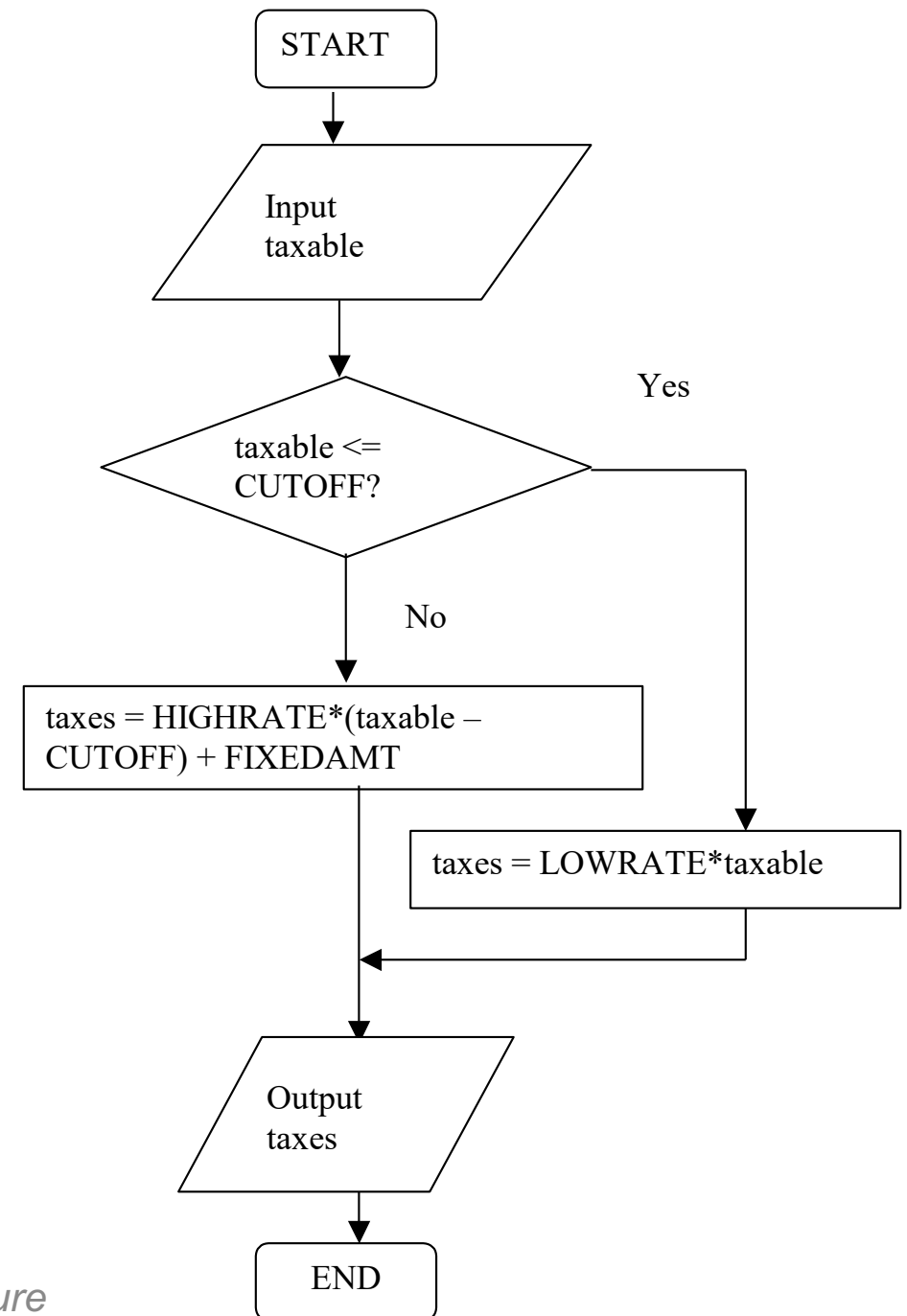


- The *if-else* statement directs the computer to select a sequence of one or more statements based on the result of a comparison.
- The syntax:

```
if (<conditional expression>) {  
    <statements-1>;  
}  
else {  
    <statements-2>;  
}
```
- If only 1 statement, we can skip the block `{ }` for *if-else*

Example 3.3.1

- We construct a C++ program for determining income taxes.
- Assume that these taxes are assessed at 2% of taxable incomes less than or equal to \$20,000.
- For taxable income greater than \$20,000, taxes are 2.5% of the income that exceeds \$20,000 plus a fixed amount of \$400.



Example 3.3.1

```
#include <iostream>
#include <iomanip>
using namespace std;
// lower tax rate
const float LOWRATE = 0.02;
// higher tax rate
const float HIGHRATE = 0.025;
// cut off for low rate
const float CUTOFF = 20000.0;
const float FIXEDAMT = 400;
int main()
{
    float taxable, taxes;
    cout << "Please type the taxable income: ";
    cin >> taxable;
```

```
    if (taxable <= CUTOFF)
        taxes = LOWRATE * taxable;
    else
        taxes = HIGHRATE *
            (taxable - CUTOFF) + FIXEDAMT;

    // set output format
    cout << setiosflags(ios::fixed)
        << setiosflags(ios::showpoint)
        << setprecision(2);
    cout << "Taxes are $ " << taxes
        << endl;
    return 0;
```

Example 3.3.1

- `setiosflags`: this manipulator is used to control different input and output settings.
 - `setioflag(ios::fixed)` means the output field will use conventional fixed-point decimal notation. (ex: 12.568)
 - `setiosflag(ios::showpoint)` means the output field will show the decimal point for floating point number.
 - `setiosflag(ios::scientific)` means the output field will use exponential notation. (1.0e-10)
- The results of the above program:
Please type in the taxable income: 10000
Taxes are \$ 200

Please type in the taxable income: 30000
Taxes are \$ 650

Block Scope

- All statements within a compound statement constitute a single **block** of code, and any variable declared within such a block only is valid within the block.
- The location within a program where a variable can be used formally referred to as the **scope** of the variable.

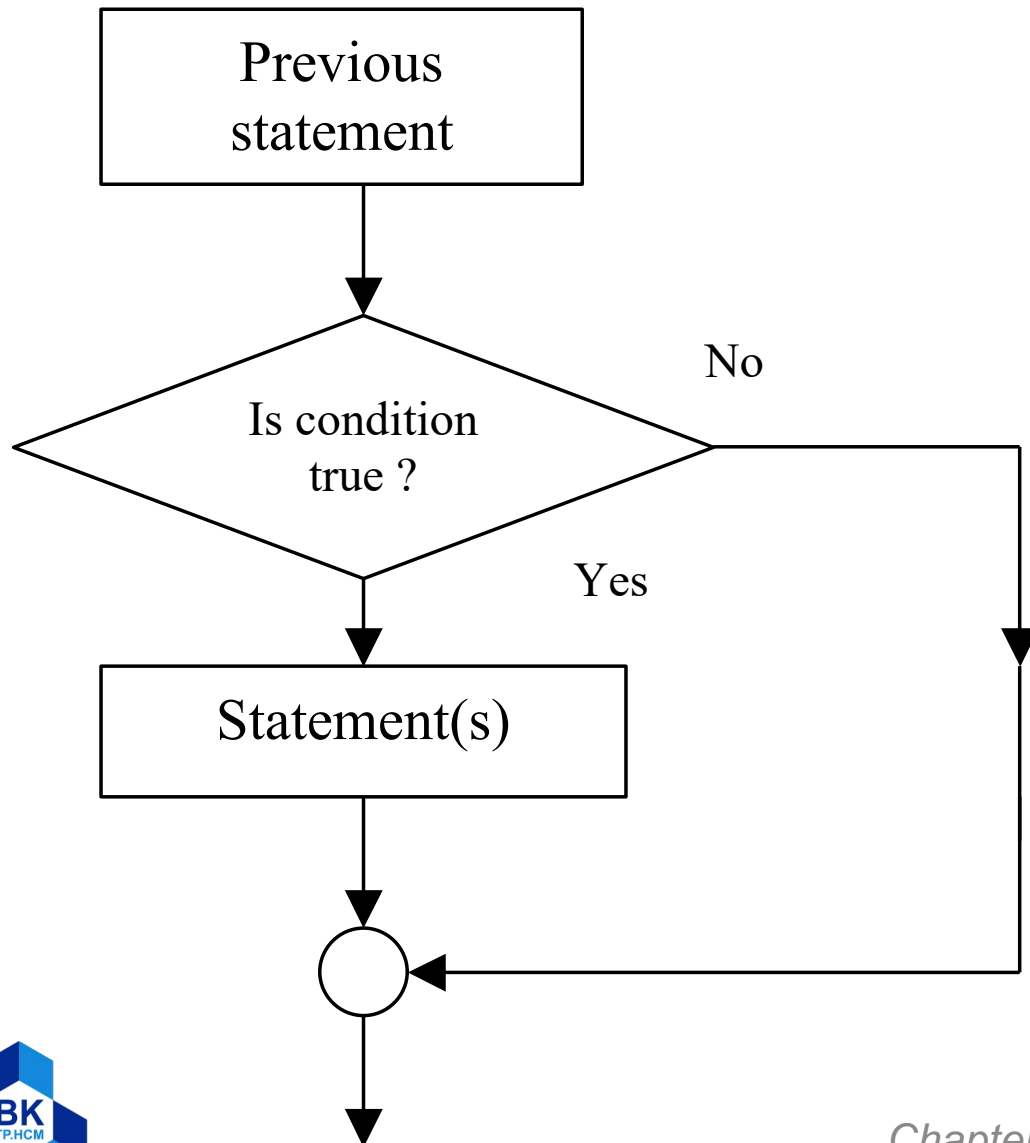
Example 3.3.2

```
{ // start of outer block
    int a = 25;
    int b = 17;
    cout << "The value of a is " << a << " and b is " << b << endl;
    { // start of inner block
        float a = 46.25;
        int c = 10;
        cout << "a is now " << a << " b is now " << b
            << " and c is " << c << endl;
    }
    cout << "a is now " << a << " b is now " << b << endl;
} // end of outer block
```

- The output is

```
The value of a is 25 and b is 17
a is now 46.25 b is now 17 and c is 10
a is now 25 b is now 17
```

One-way Selection



- A useful modification of the if-else statement involves omitting the else part of the statement. In this case, the if statement takes a shortened format:

```
if (<conditional expression>) {  
    <statements>;  
}
```

Example 3.3.3

- The following program displays an error message for the grades that is less than 0 or more than 100.

```
#include <iostream>
using namespace std;
int main()
{
    int grade = 0;
    cout << "Please enter a grade: ";
    cin >> grade;
    if (grade < 0 || grade > 100)
        cout << "\nThe grade is not valid\n";
    return 0;
}
```

Content

-
- | | |
|---|---|
| 1 | Program Flow of Control |
| 2 | Boolean data type and boolean expressions |
| 3 | The if-else statement |
| 4 | Nested if statement |
| 5 | <i>switch</i> statement |
-

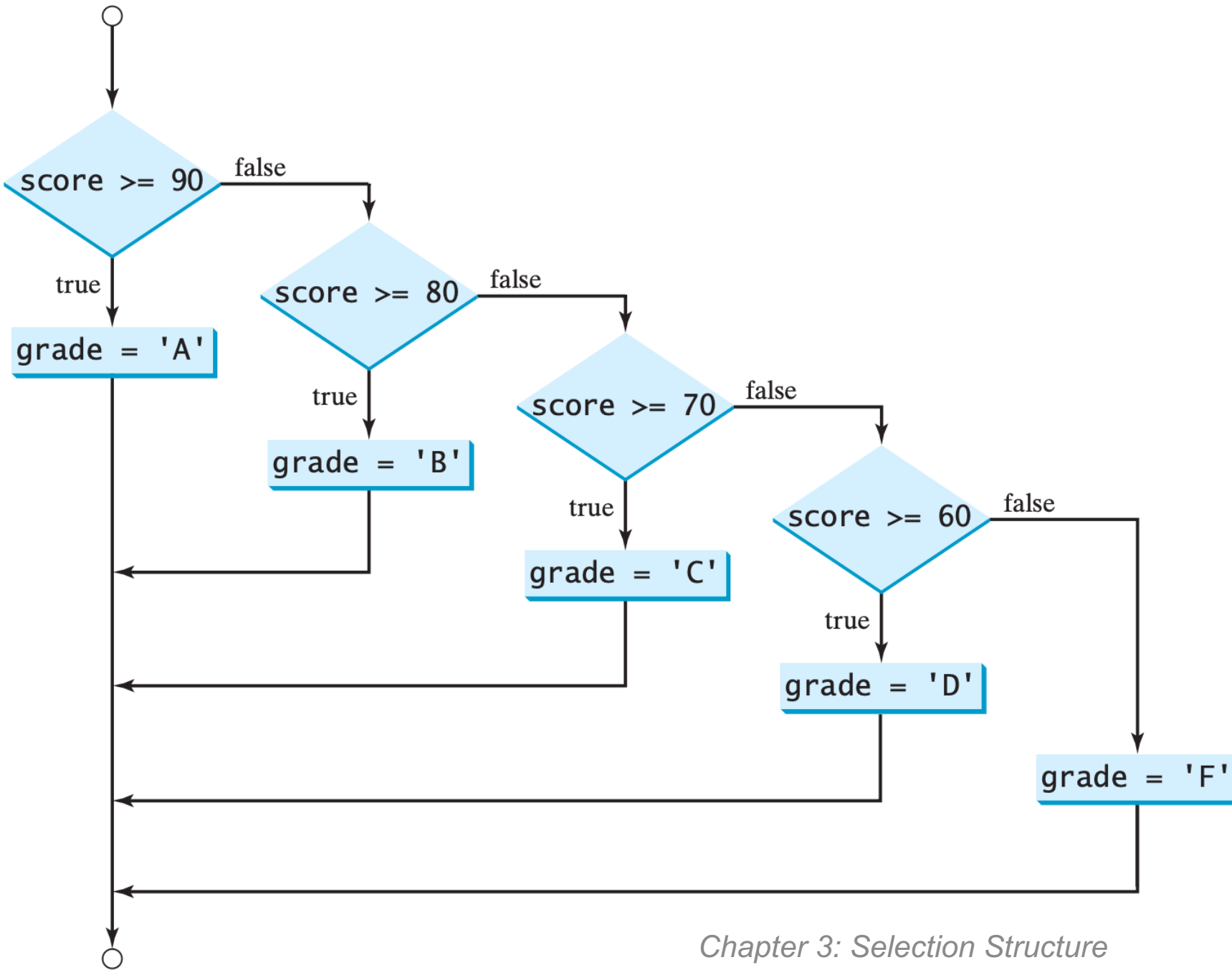
Nested *if* statement

```
if (<boolean-expression>) {  
    <statement(s)-1>;  
}  
else if (<boolean-expression-2>) {  
    <statement(s)-2>;  
}  
else {  
    <statement(s)-default>;  
}
```

- The inclusion of one or more *if* statement within an existing *if* statement is called a *nested if statement*.
- The *if-else chain*: When an *if* statement is included in the else part of an existing *if* statement, we have an *if-else chain*.

Example 3.4.1

```
if (score >= 90.0)
    grade = 'A';
else if (score >= 80.0)
    grade = 'B';
else if (score >= 70.0)
    grade = 'C';
else if (score >= 60.0)
    grade = 'D';
else
    grade = 'F';
```



Example 3.4.1

- This program can solve quadratic equation.
- The output of the above program:

Enter the coefficients of the equation:

1 5 6

x1 = -2.0 x2 = -3.0

```
#include <iostream>
#include <cmath>
#include <iomanip>
using namespace std;
int main() {
    double a, b, c, del, x1, x2;
    cout << "Enter the coefficients of the equation: \n";
    cin >> a >> b >> c;
    del = b * b - 4.0 * a * c;
    if (del == 0.0) {
        x1 = x2 = -b / (2 * a);
        cout << "x1 = " << x1 << setw(20) << "x2 = "
              << x2 << endl;
    }
    else if (del > 0.0) {
        x1 = (-b + sqrt(del)) / (2 * a);
        x2 = (-b - sqrt(del)) / (2 * a);
        cout << "x1 = " << x1 << setw(20) << "x2 = "
              << x2 << endl;
    }
    else // del < 0
        cout << "There is no solution\n";
    return 0;
}
```

Content

-
- | | |
|---|---|
| 1 | Program Flow of Control |
| 2 | Boolean data type and boolean expressions |
| 3 | The if-else statement |
| 4 | Nested if statement |
| 5 | switch statement |
-

The *switch* statement

- The *switch* statement controls program flow by executing a set of statements depending on the value of an expression.
- The syntax for the *switch* statement:

```
switch (<expression>) {  
    case <label1>:  
        <statement(s)-1>;  
        break;  
    case <label2>:  
        <statement(s)-2>;  
        break;  
    default:  
        <statement(s)-3>;  
}
```

Note: The value of expression must be an *integer* data type, which includes the *char*, *int*, *long int*, and *short* data types.

Execution of the *switch* statement

- The *expression* in the *switch* statement must *evaluate to an integer* result.
- The *switch* expression's value is *compared* to each of these case values *in the order* in which these values are listed *until a match* is found.
- When a *match* occurs, *execution* begins with *the statement following the match*.
- If the value of the *expression does not match* any of the case values, no statement is executed unless the keyword *default* is encountered.
 - If the value of the expression does not match any of the case values, program execution begins with the statement following the word *default*.

break statements in the *switch* statement

- The *break* statement is used to identify the end of a particular *case* and causes an *immediate exit* from the *switch* statement.
- If the *break* statements are omitted, *all cases following* the *matching case value*, including the *default* case, *are executed*.

Example 3.5.1

```
#include <iostream>
using namespace std;
int main() {
    int num1, num2;
    char op;
    cout << "Enter two number: ";
    cin >> num1 >> num2;
    cout << "Enter an operator: ";
    cin >> op;
    switch (op) {
        case '+':
            cout << "Result: "
                 << num1 + num2 << endl;
            break;
        case '-':
            cout << "Result: "
                 << num1 - num2 << endl;
            break;
```

```
        case '*':
            cout << "Result: "
                 << num1 * num2 << endl;
            break;
        case '/':
            if (num2 != 0)
                cout << "Result: "
                     << num1 / num2 << endl;
            else
                cout << "Division by zero!"
                     << endl;
            break;
        default:
            cout << "Invalid operator!"
                 << endl;
    }
    return 0;
}
```

Content

-
- | | |
|---|---|
| 1 | Program Flow of Control |
| 2 | Boolean data type and boolean expressions |
| 3 | The if-else statement |
| 4 | Nested if statement |
| 5 | switch statement |
-



VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Thank you!